ToolShare

Peer-to-Peer Tool Rental Platform

Phase 4: System Analysis & Design

Team
Dareen
Ahmed
Mahmoud Sobhy
Menna
Mahmoud Elsayed

Stack: MongoDB, Express, React + TypeScript, Node.js

Contents

#	Section
1	Introduction and Scope
2	Requirements Summary
3	Use Case Diagram
4	Activity Diagram: Booking a Tool
5	Sequence Diagram: Create Booking Request
6	Entity Relationship Diagram (ERD)
7	System Architecture
8	UI/UX Design
9	API Documentation
10	Business Rules, Assumptions and Open Decisions
11	Deliverables and Repository Structure

1. Introduction and Scope

ToolShare is a web platform where people lend and rent tools to their neighbors. Owners list tools they rarely use; renters browse, pick dates and send a booking request; owners accept or reject it. This document covers the analysis and design of the system before implementation: use cases, behavior (activity and sequence), data model, architecture, user interface and API.

1.1 Technology stack

Layer	Technology
Front-End	React + TypeScript
Back-End	Node.js + Express (REST API)
Database	MongoDB (Users, Tools, Bookings, Reviews collections)
Authentication	JWT
Design	Figma

1.2 Team responsibilities

Member	Responsibility
Dareen	Figma UI/UX + User Flow
Ahmed	System Architecture + Front-End structure
Mahmoud Sobhy	Tool pages + Search/Filter flow
Menna	Authentication + Booking flow
Mahmoud Elsayed	ERD + Database + API Documentation

1.3 MVP scope

- User registration and login (JWT).
- Browse, search and filter tools; view tool details.
- Owners create, edit and deactivate their listings.
- Renters send booking requests with dates; owners accept or reject.
- Renters track their bookings and cancel pending ones.
- Renters review a tool after a completed booking.

Out of MVP (shown in the UI mockups as future features): online payment and deposits, insurance/guarantee, in-app messaging, favorites, distance-based search, draft listings, and a full admin panel.

2. Requirements Summary

2.1 Functional requirements

ID	Requirement	Actor
FR-1	Register a new account and log in	Guest
FR-2	Browse tools, search by keyword, filter by category and price, sort results	Guest / User
FR-3	View tool details, photos, owner info and reviews	Guest / User
FR-4	Create a booking request for a date range	Registered User
FR-5	Prevent booking dates that overlap an accepted booking	System

ID	Requirement	Actor
FR-6	Create, edit, pause and delete own tool listings	Tool Owner
FR-7	Accept or reject incoming booking requests	Tool Owner
FR-8	Track own bookings by status and cancel a pending request	Registered User
FR-9	Submit a rating and comment after a completed booking	Registered User
FR-10	Manage users and listings (moderation)	Administrator (if in MVP)

2.2 Non-functional requirements

- **Security:** passwords hashed (bcrypt); JWT for protected routes; role-based authorization; input validation on every endpoint.
- **Usability:** responsive layout; clear status labels for bookings.
- **Performance:** paginated listings; indexes on category, owner and tool/date fields.
- **Maintainability:** layered architecture (routes, controllers, services, models).

3. Use Case Diagram

Four actors interact with the system. Tool Owner and Administrator are specializations of a Registered User.

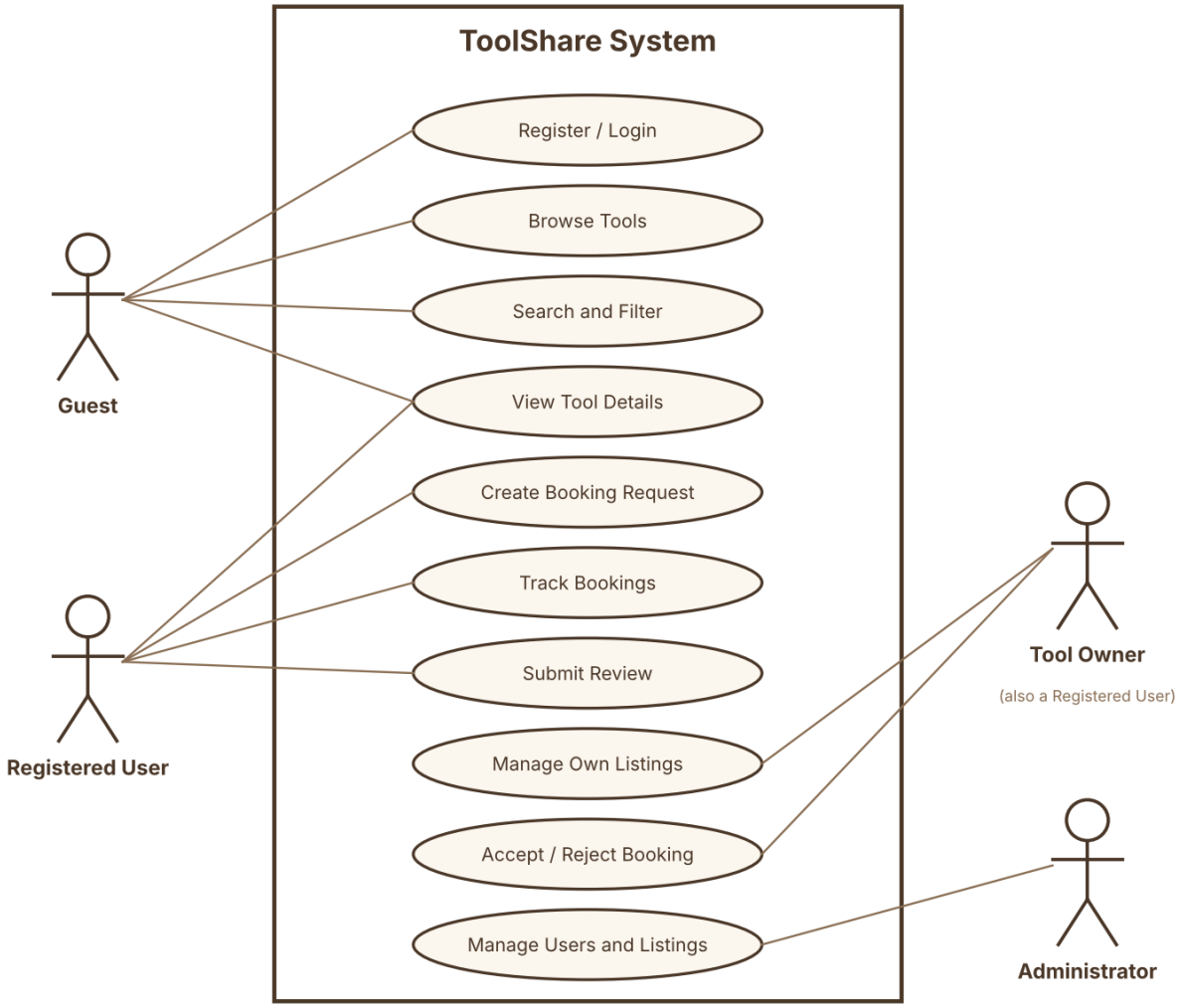


Figure 1: Use Case Diagram

Actor	Can do
Guest	Register / log in, browse tools, search and filter, view tool details
Registered User (Renter)	Everything a Guest can, plus create booking requests, track and cancel bookings, submit reviews
Tool Owner	Manage own listings, accept or reject booking requests
Administrator	Manage users and listings

4. Activity Diagram: Booking a Tool

The flow from selecting a tool to the owner's decision, including the validation loops for invalid and unavailable dates.

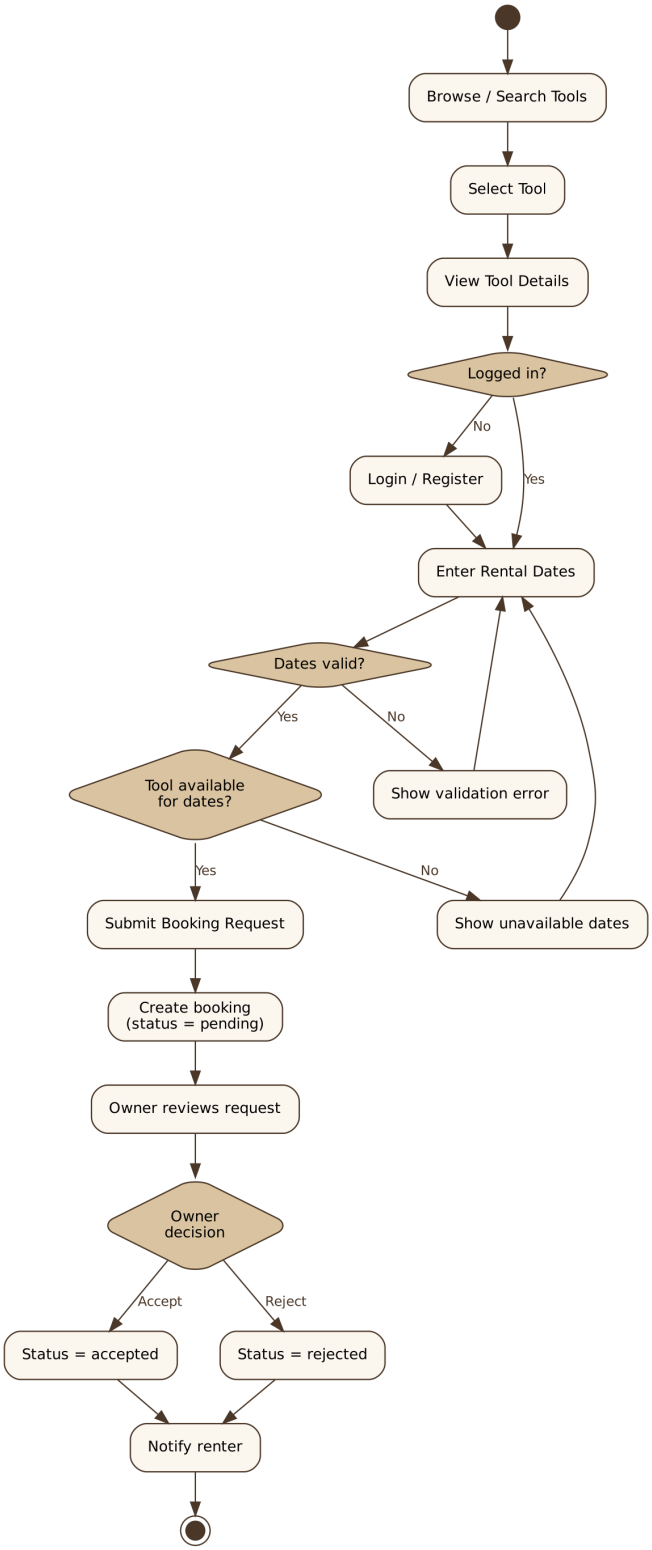


Figure 2: Activity Diagram

5. Sequence Diagram: Create Booking Request

The renter's request passes through the React front-end to the Express API, which validates it and checks existing accepted bookings in MongoDB. If the dates are free, a booking with status **pending** is created (201). Otherwise the API answers **409 Conflict**.

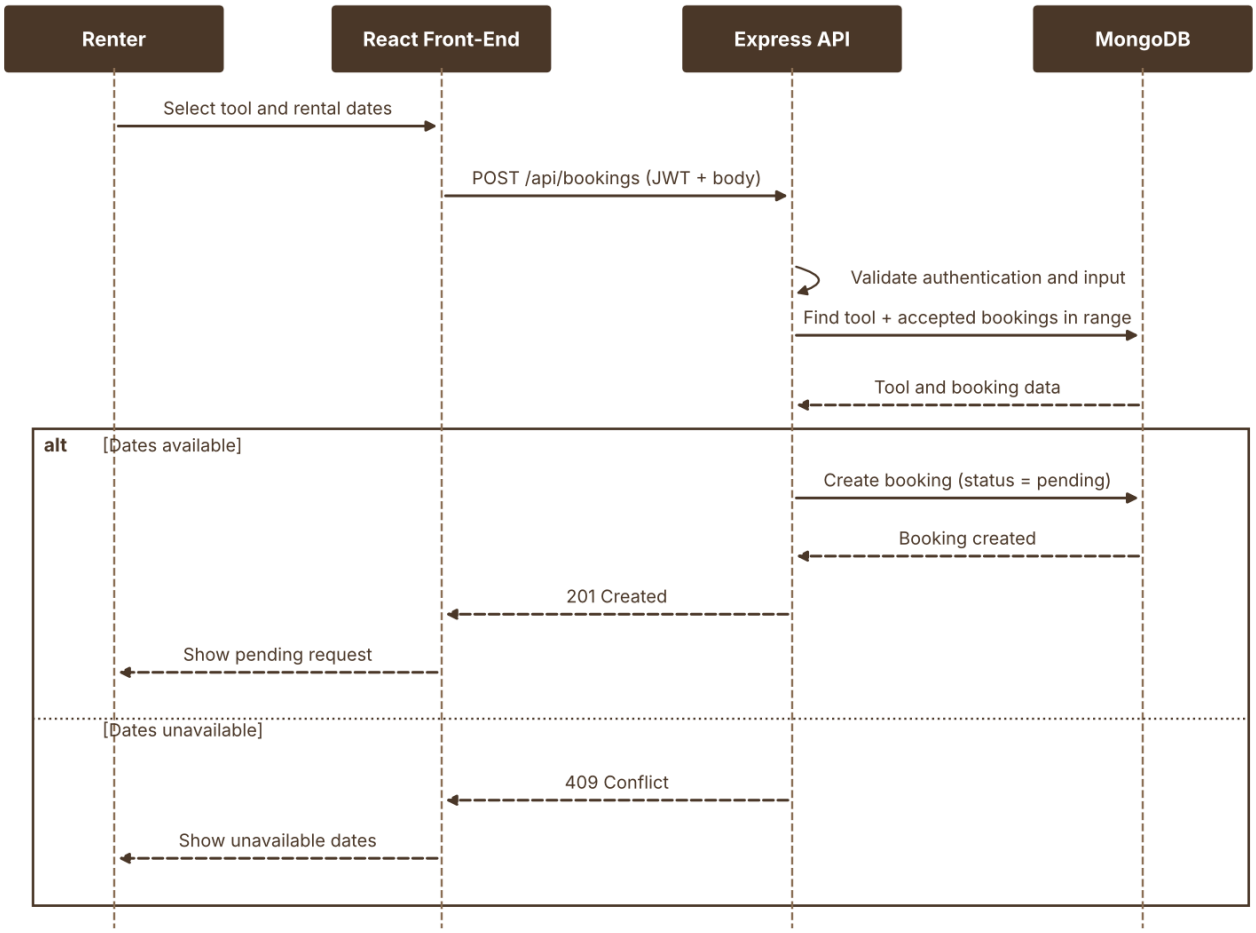


Figure 3: Sequence Diagram

6. Entity Relationship Diagram (ERD)

The project uses MongoDB, so the ERD shows logical relationships between collections (references by ObjectId), not SQL foreign-key constraints.

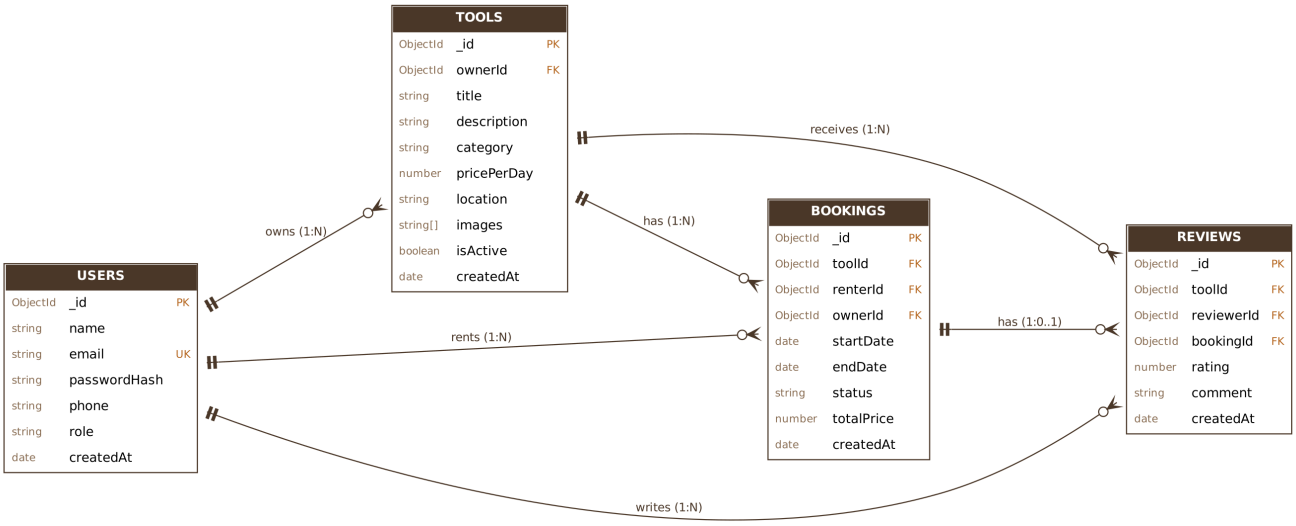


Figure 4: ERD

6.1 Relationships

Relationship	Meaning
User 1 : N Tools	A user can list many tools
User 1 : N Bookings (as renter)	A user can make many bookings
Tool 1 : N Bookings	A tool can have many booking requests
User 1 : N Reviews	A user can write many reviews
Tool 1 : N Reviews	A tool can receive many reviews
Booking 1 : 0..1 Review	A booking has at most one review

6.2 Data dictionary

users

Field	Type	Notes
_id	ObjectId	Primary key
name	String	Required
email	String	Unique, required
passwordHash	String	bcrypt hash; never returned by the API
phone	String	Optional
role	String	user | admin (owner is a user who has listed tools)
createdAt	Date	Default: now

tools

Field	Type	Notes
_id	ObjectId	Primary key
ownerId	ObjectId	Ref: users._id
title	String	Required
description	String	Max 1,000 chars
category	String	Power Tools, Gardening, Construction, Home Improvement, Automotive, Hand Tools
pricePerDay	Number	> 0
location	String	Neighborhood / pickup district (exact address is not public)
images	String[]	Up to 6 image URLs
isActive	Boolean	false when paused or deleted
createdAt	Date	Default: now

bookings

Field	Type	Notes
_id	ObjectId	Primary key
toolId	ObjectId	Ref: tools._id
renterId	ObjectId	Ref: users._id
ownerId	ObjectId	Ref: users._id (copied from the tool for faster owner queries)
startDate / endDate	Date	endDate >= startDate; startDate not in the past
status	String	pending | accepted | rejected | completed | cancelled
totalPrice	Number	days x pricePerDay, computed by the server
createdAt	Date	Default: now

reviews

Field	Type	Notes
_id	ObjectId	Primary key
toolId	ObjectId	Ref: tools._id

Field	Type	Notes
reviewerId	ObjectId	Ref: users._id
bookingId	ObjectId	Ref: bookings._id (unique: one review per booking)
rating	Number	1 to 5
comment	String	Optional
createdAt	Date	Default: now

7. System Architecture

A three-layer architecture. The React client talks to the Express REST API over HTTPS with JSON; the API applies authentication, validation and business rules, and persists data in MongoDB.

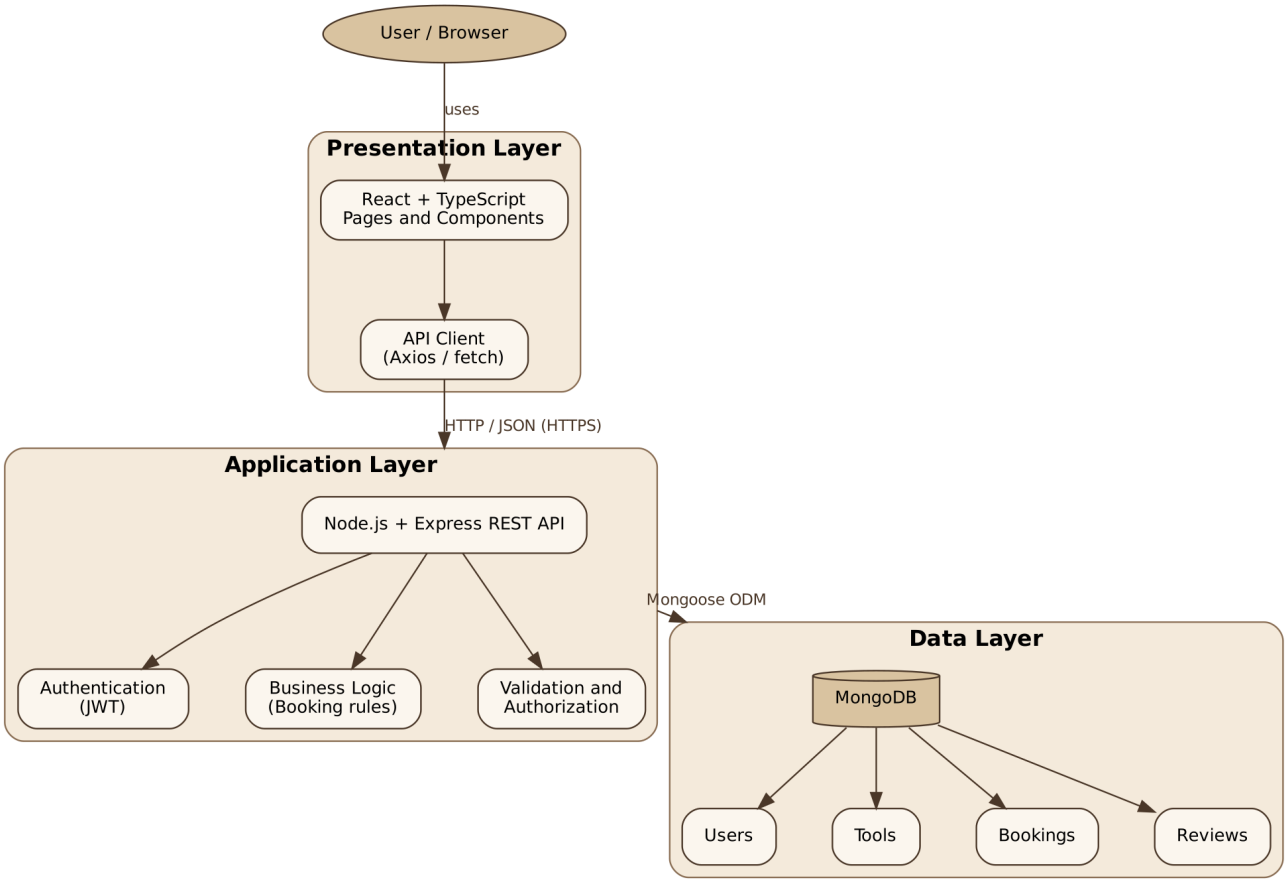


Figure 5: System Architecture

Layer	Responsibility
Presentation	Pages, components, routing, form validation, API client with JWT in the Authorization header
Application	REST endpoints, JWT authentication, role checks (owner/admin), validation, booking-conflict logic
Data	MongoDB collections accessed through Mongoose models

8. UI/UX Design

The screens were designed in Figma. A shared warm brown/beige palette, one typeface family and reusable cards keep the screens consistent.

8.1 User flow

Step	Screen	Action
1	Home	Search or open a category
2	Explore Tools	Filter, sort, open a tool
3	Tool Details	Pick dates, press Request to Book (login required)
4	Login / Sign Up	Authenticate, return to the tool
5	My Bookings	Track status; cancel pending; review after completion
6	Owner Dashboard	Accept / Reject requests; manage listings
7	Add New Tool	Create a listing

Figma link: *(add the team's Figma URL here and in Figma-Link.txt)*

8.2 Home Page

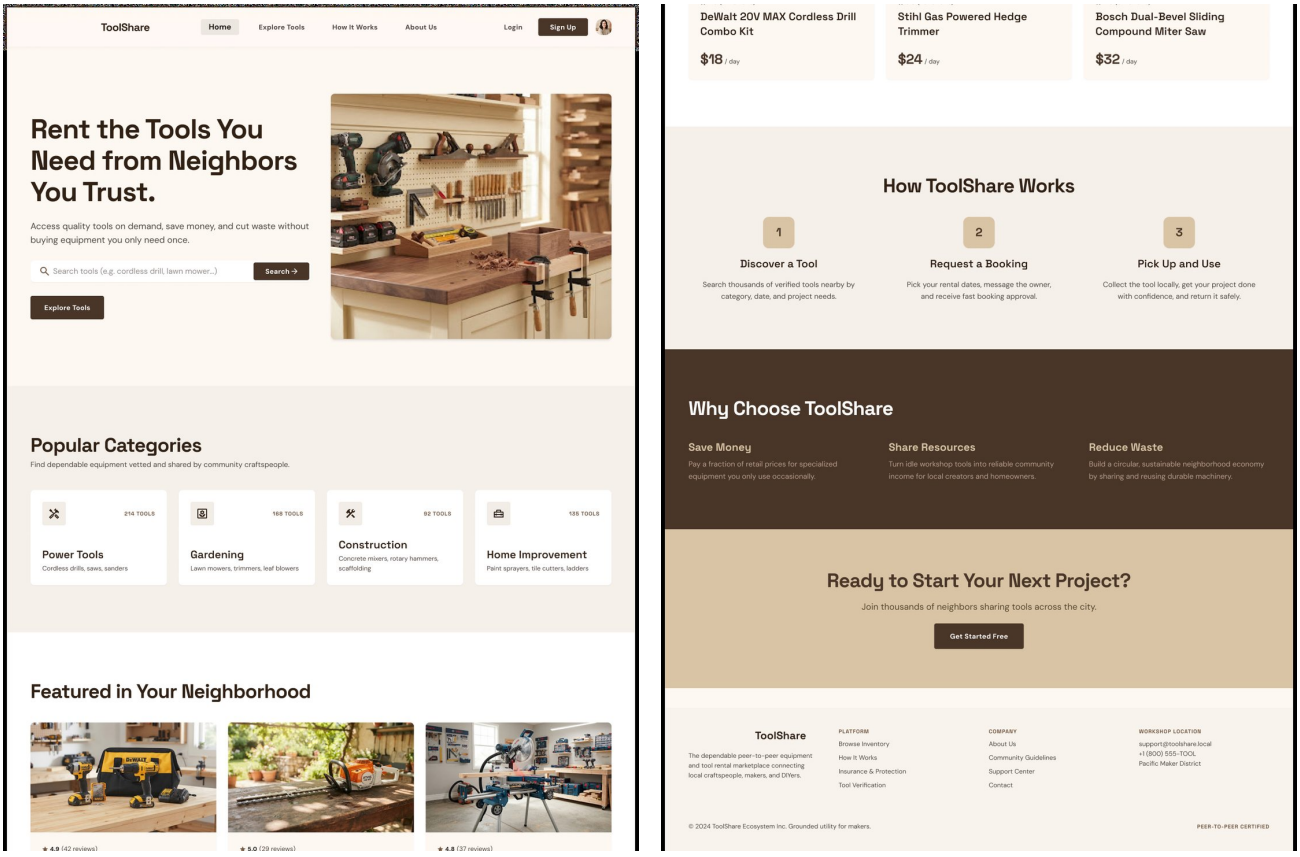


Figure: Home Page (top half left, bottom half right)

- Hero with search box and Explore Tools button; popular categories; featured tools; How It Works; call to action.

8.3 Explore Tools (Tools Listing)

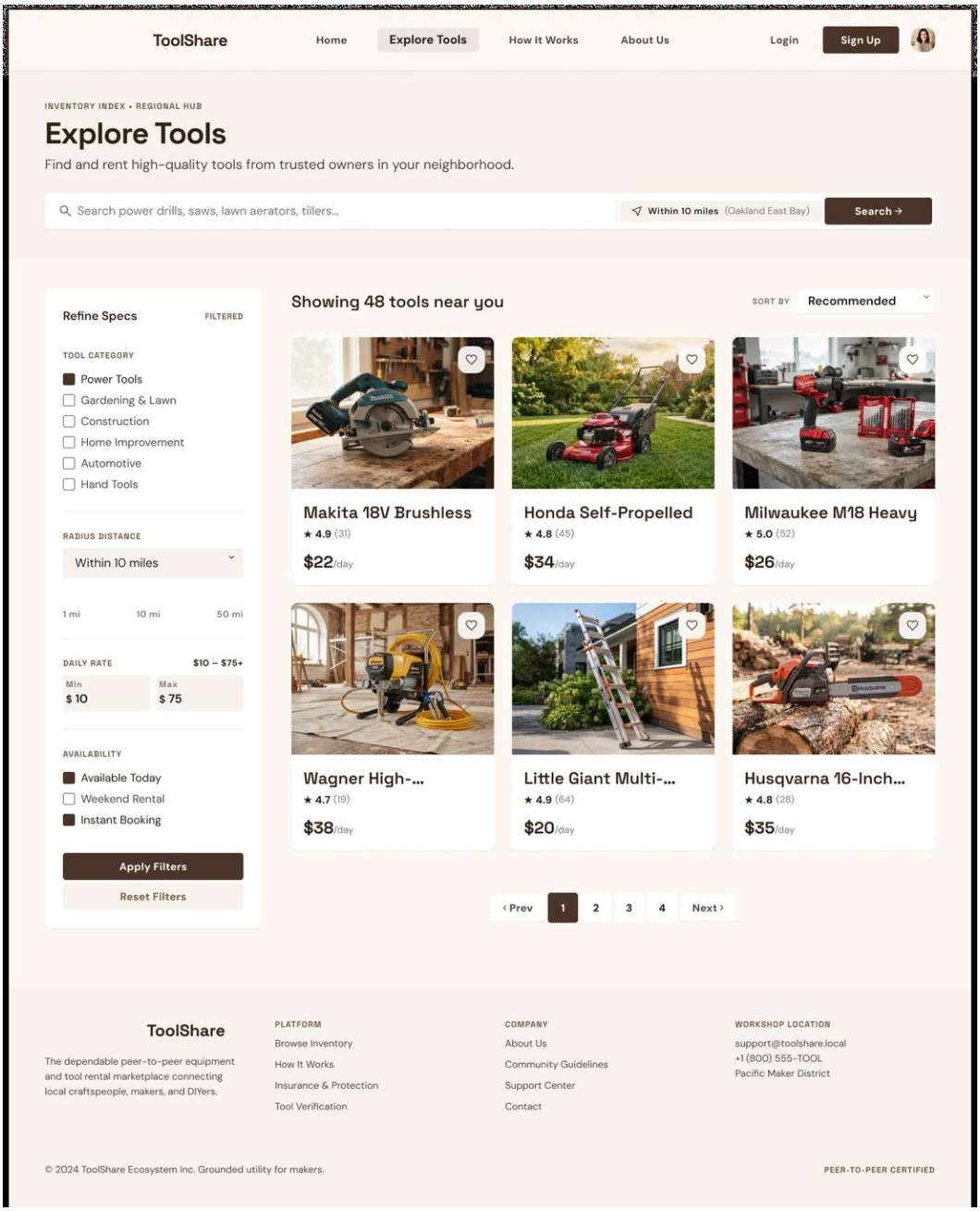


Figure: Explore Tools (Tools Listing)

- Search bar, category / price / availability filters, sort menu, card grid and pagination.
- MVP filters: keyword, category, price range. Distance, Weekend and Instant Booking are future features.

8.4 Tool Details

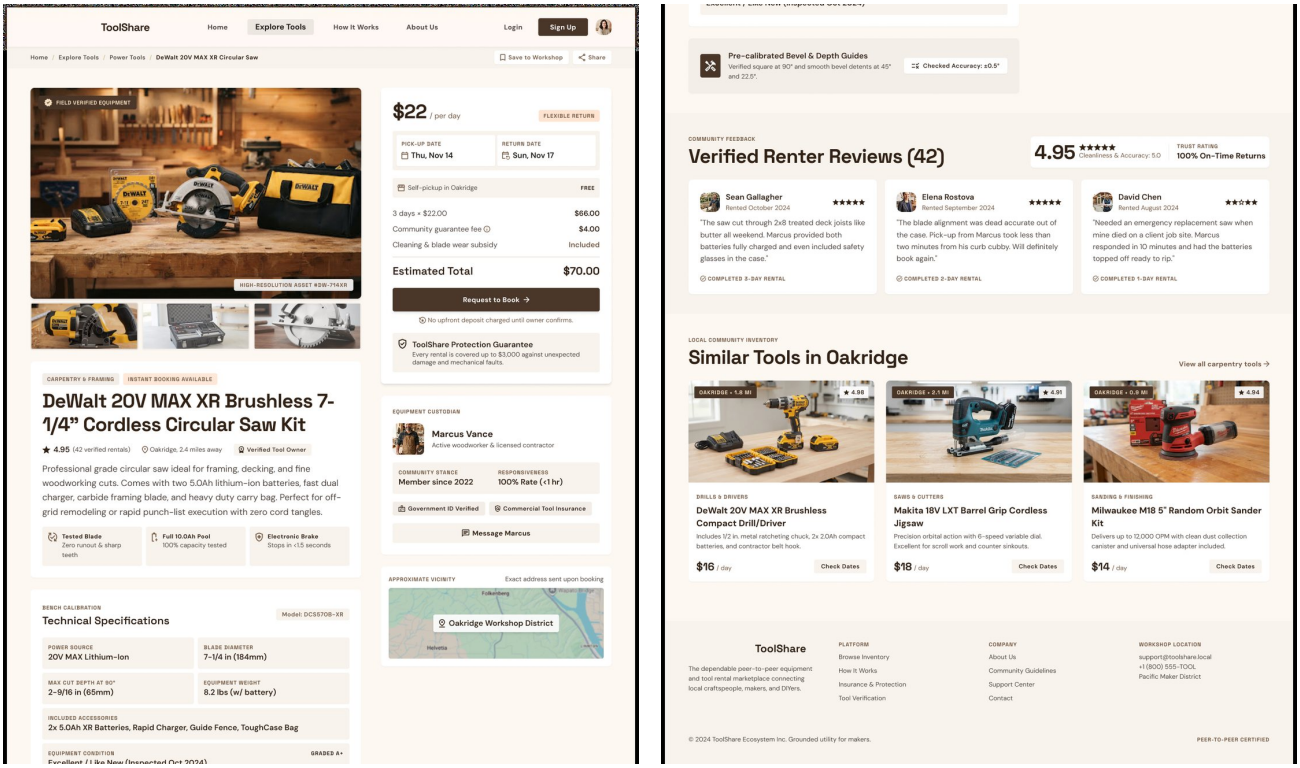


Figure: Tool Details (top half left, bottom half right)

- Photo gallery, description, specifications, owner card, date pickers and price estimate, reviews, similar tools.
- The Request to Book action creates a booking with status pending. Price = days x daily rate.

8.5 Login and Sign Up

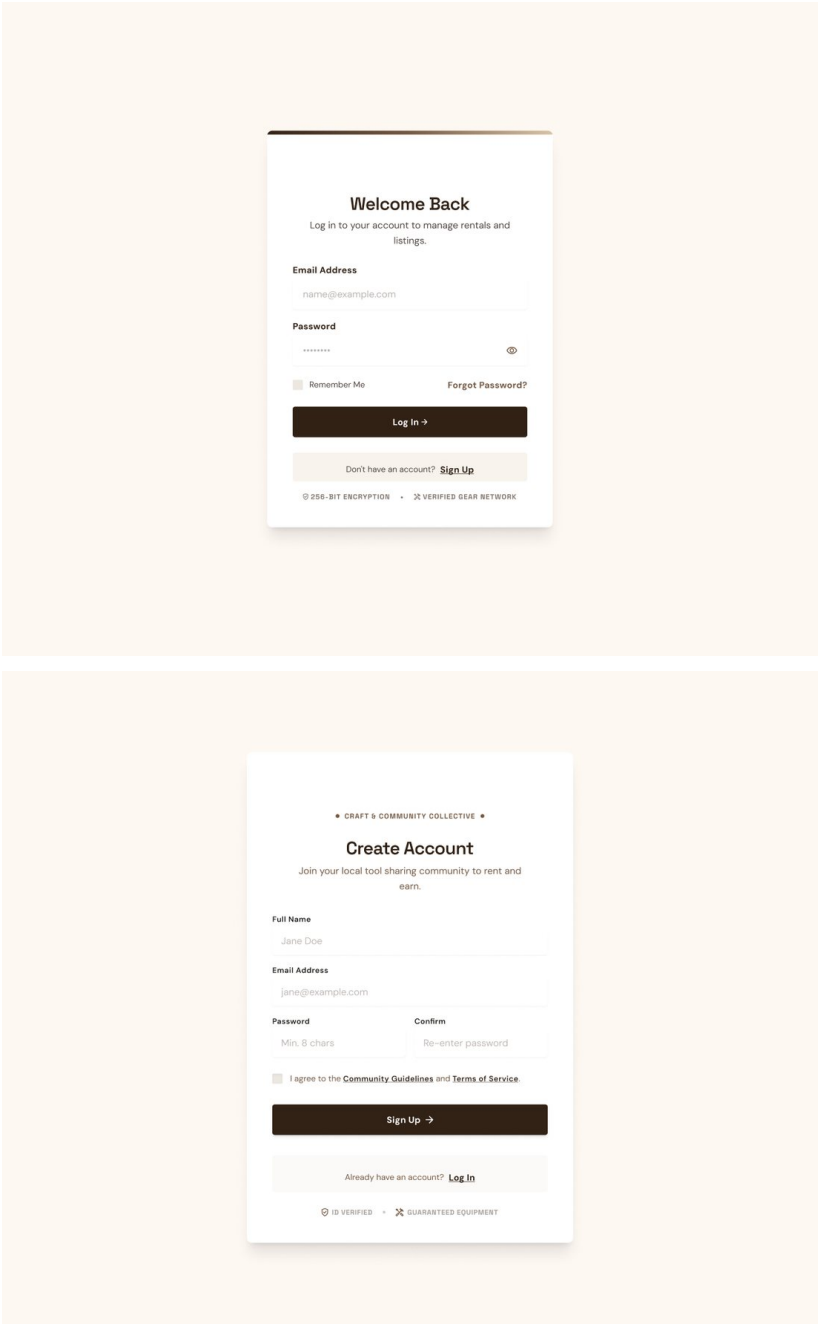


Figure: Login and Sign Up

- Login: email, password, show/hide password, link to Sign Up. Sign Up: name, email, password and confirmation, terms checkbox.

8.6 Add / Edit Tool

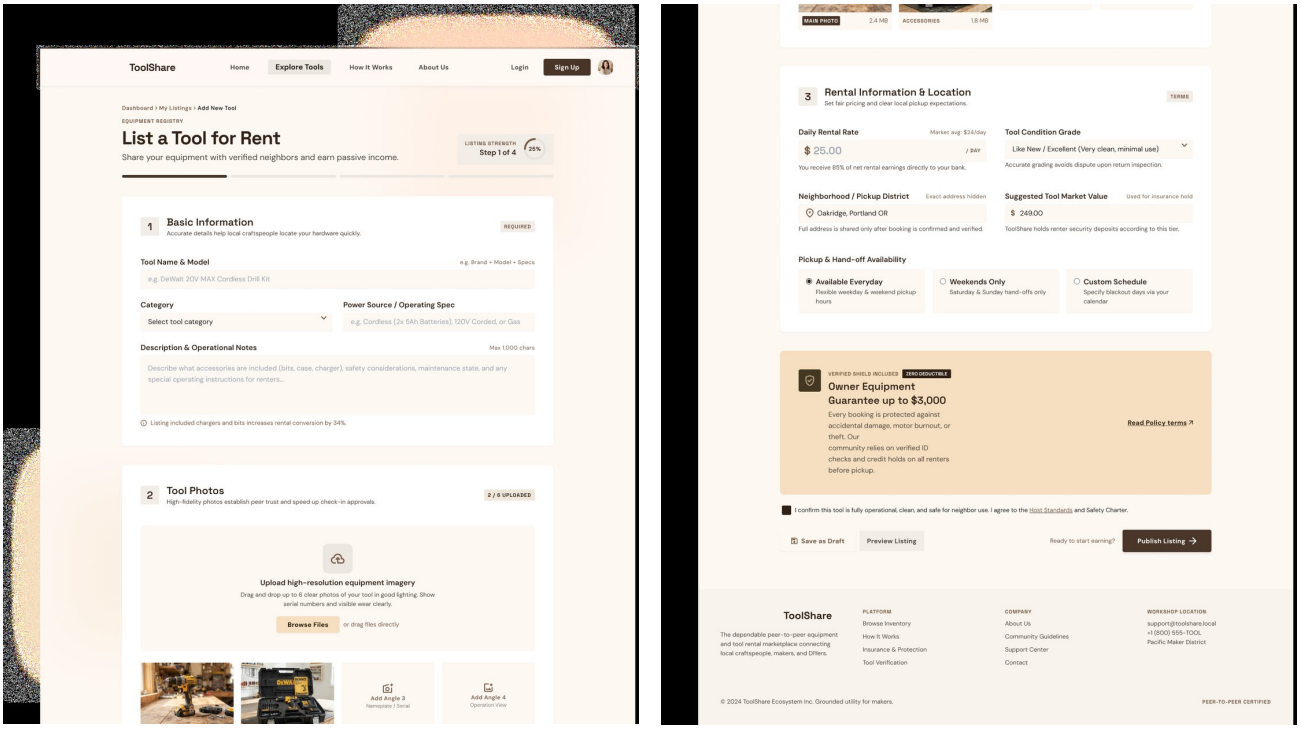


Figure: Add / Edit Tool (top half left, bottom half right)

- Three sections: basic info, photos (up to 6), rental info and location. The same form is reused for editing with fields pre-filled.
- Fields beyond the ERD (condition grade, market value, power source, availability schedule) are future features.

8.7 My Bookings

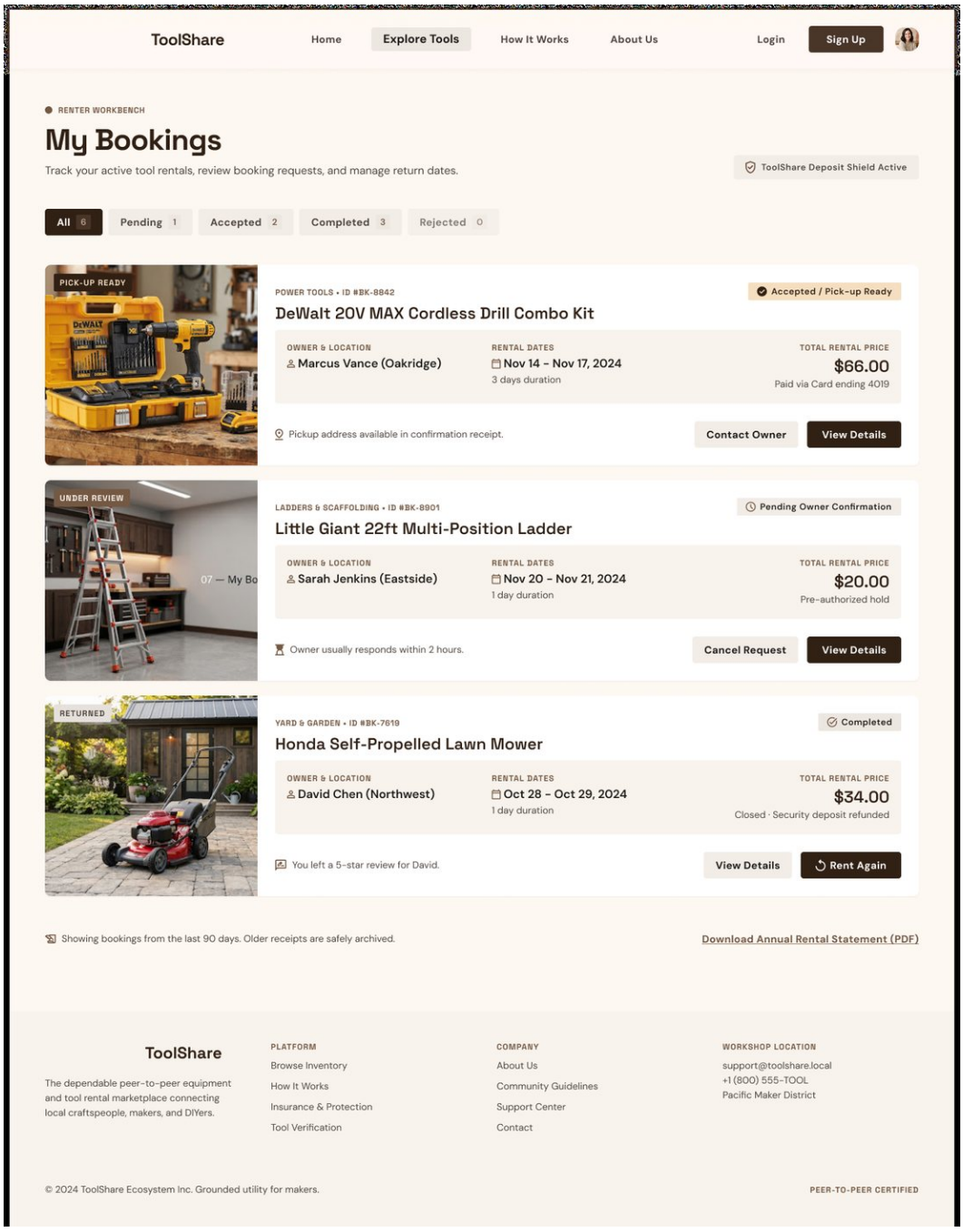


Figure: My Bookings

- Status tabs: All, Pending, Accepted, Completed, Rejected. Add a Cancelled tab to match the booking statuses.
- Actions: Cancel Request (pending), Contact Owner / View Details (accepted), Rent Again (completed).

8.8 Owner Dashboard

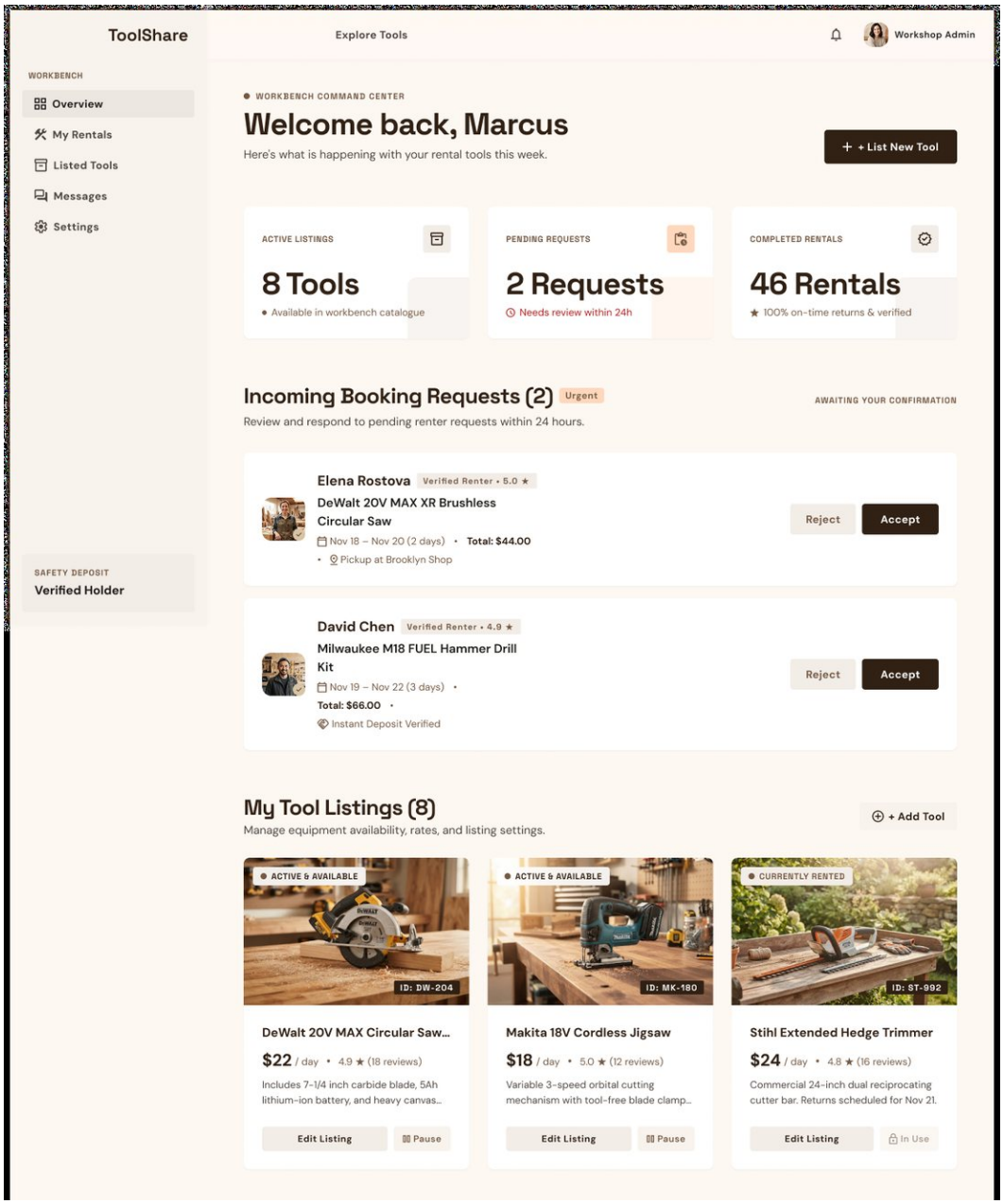


Figure: Owner Dashboard

- Summary cards, incoming booking requests with Accept / Reject, and the owner's listings with Edit and Pause.

9. API Documentation

Base URL: `/api`. Format: JSON. Protected endpoints require the header `Authorization: Bearer <JWT>`.

9.1 Endpoint summary

Method	Endpoint	Description	Access
POST	/auth/register	Create an account	Public
POST	/auth/login	Log in, receive JWT	Public
GET	/auth/me	Current user profile	User
GET	/tools	List tools (search, category, minPrice, maxPrice, sort, page, limit)	Public
GET	/tools/:id	Tool details with owner summary	Public
GET	/tools/mine	Tools listed by the current user	Owner
POST	/tools	Create a listing	User
PUT	/tools/:id	Update a listing (owner only)	Owner
PATCH	/tools/:id/active	Pause or activate a listing	Owner
DELETE	/tools/:id	Delete a listing (no pending/accepted bookings)	Owner / Admin
GET	/tools/:id/reviews	Reviews of a tool	Public
POST	/bookings	Create a booking request (status = pending)	User
GET	/bookings/mine	Bookings made by the current user (?status=)	User
GET	/bookings/incoming	Booking requests on the user's tools (?status=)	Owner
PATCH	/bookings/:id/status	Owner: accepted | rejected | completed	Owner
PATCH	/bookings/:id/cancel	Renter cancels a pending booking	Renter
POST	/reviews	Review a completed booking	Renter
GET	/dashboard/owner	Counts: active listings, pending requests, completed rentals	Owner
GET	/admin/users	List users (if admin is in MVP)	Admin
DELETE	/admin/tools/:id	Remove any listing (if admin is in MVP)	Admin

9.2 Status codes

Code	Meaning
200 / 201	Success / resource created
400	Validation error (bad input or dates)
401	Missing or invalid token
403	Not allowed (wrong role or not the owner)
404	Resource not found
409	Conflict: dates overlap an accepted booking, or duplicate email / review

9.3 Request / response examples

POST /auth/register

```
Request
{ "name": "Jane Doe", "email": "jane@example.com", "password": "min8chars", "phone": "01000000000" }

Response 201
{ "token": "<JWT>", "user": { "_id": "665f...", "name": "Jane Doe", "email": "jane@example.com", "role": "user" } }
```

POST /auth/login

```
Request
{ "email": "jane@example.com", "password": "min8chars" }

Response 200
{ "token": "<JWT>", "user": { "_id": "665f...", "name": "Jane Doe", "role": "user" } }
Response 401
{ "message": "Invalid email or password" }
```

GET /tools?search=drill&category=Power Tools&minPrice=10&maxPrice=75&page=1&limit=9

```
Response 200
{ "page": 1, "limit": 9, "total": 48,
  "tools": [ { "_id": "666a...", "title": "DeWalt 20V MAX Drill Combo Kit", "category": "Power Tools",
               "pricePerDay": 18, "location": "Oakridge", "images": ["https://.../1.jpg"],
               "rating": 4.9, "reviewsCount": 42 } ] }
```

POST /tools (Owner)

```
Request
{ "title": "DeWalt 20V MAX XR Circular Saw", "description": "Includes 2 batteries and case",
  "category": "Power Tools", "pricePerDay": 22, "location": "Oakridge", "images": ["https://.../1.jpg"] }

Response 201
{ "_id": "666b...", "ownerId": "665f...", "title": "DeWalt 20V MAX XR Circular Saw", "isActive": true }
```

POST /bookings (Renter)

```
Request
{ "toolId": "666b...", "startDate": "2026-11-14", "endDate": "2026-11-17" }

Response 201
{ "_id": "667c...", "toolId": "666b...", "renterId": "665f...", "ownerId": "664e...",
  "startDate": "2026-11-14", "endDate": "2026-11-17", "status": "pending", "totalPrice": 88 }
Response 409
{ "message": "Tool is not available for the selected dates" }
```

PATCH /bookings/:id/status (Owner)

```
Request
{ "status": "accepted" }

Response 200
{ "_id": "667c...", "status": "accepted" }
Response 403
{ "message": "Only the tool owner can change this booking" }
```

POST /reviews (Renter)

```
Request
{ "bookingId": "667c...", "rating": 5, "comment": "Worked perfectly, great owner." }

Response 201
{ "_id": "668d...", "toolId": "666b...", "reviewerId": "665f...", "bookingId": "667c...", "rating": 5 }
Response 400
{ "message": "Only completed bookings can be reviewed" }
```

10. Business Rules, Assumptions and Open Decisions

10.1 Business rules

- A booking starts as **pending**. Only the tool owner can move it to **accepted** or **rejected**; only the renter can **cancel** a pending booking.
- The owner marks an accepted booking **completed** after the tool is returned.
- Dates may not overlap an **accepted** booking of the same tool. Pending requests do not block dates.
- A user cannot book their own tool.
- Only a renter with a **completed** booking can review, once per booking; rating is 1 to 5.
- totalPrice is always computed by the server (number of days x pricePerDay).

10.2 Open decisions for the team

#	Decision	Proposed answer
1	Is the Administrator part of the MVP?	If not, remove it from the Use Case and drop the /admin endpoints.
2	Which UI features are really implemented?	Keep keyword, category and price filters; mark payment, insurance, messaging, favorites, distance and instant booking as future work.
3	Booking statuses	pending, accepted, rejected, completed, cancelled. Add the Cancelled tab in My Bookings.
4	Roles in the UI	Use a single user role plus admin. Remove the 'Workshop Admin' label from the owner dashboard.
5	Cancel rules	Renter may cancel only while pending (an accepted booking needs the owner's agreement).
6	Currency and locations	The mockups use USD and US locations; use the project's real currency and areas.
7	ownerId in bookings	Kept for faster owner queries; it duplicates tool.ownerId.

11. Deliverables and Repository Structure

Deliverable	Status
Phase 4 Documentation (this file)	Done
Use Case, Activity, Sequence, ERD, Architecture diagrams	Done (included here and as PNG files)
UI/UX screens	Done (8 screens)
API Documentation	Done (Section 9)
Figma link	To add
Team review of diagrams and open decisions (10.2)	To do

```
Documentation/
  Phase-4/
    Phase-4-Documentation.pdf
    Use-Case-Diagram.png
    Activity-Diagram.png
    Sequence-Diagram.png
    ERD.png
    System-Architecture.png
    UI-UX/            (8 screen images)
    Figma-Link.txt
```